

DS-502-MJ

Computational Mathematics

Linear Algebra · Numerical Methods · Matrix Methods for Data Science

4 Credits · 48 Lecture Hours | Internal 30 · University 70

Complete exam-ready notes — taught from first principles, with worked examples, runnable Python, diagrams, and real data-science applications.

CO1 · Matrix algebra & linear systems

CO2 · Numerical solution methods

CO3 · Vector maps

CO4 · Mathematics in Data Science

Prepared by Sid Thaware

Portfolio & Data Science Projects

<https://portfolio-sidt0518.onrender.com>

Grounded in: **Axler** — Linear Algebra Done Right · **Lay** — Linear Algebra and Its Applications · **Hoffman & Kunze** · **Rosen** — Discrete Mathematics · **Eldén** — Matrix Methods in Data Mining

How to Use These Notes

💡 READ ME FIRST

These notes are **self-sufficient**: everything you need to pass the university exam *and* to build a real linear-algebra-for-data-science project is here. Every topic follows the same rhythm so your brain knows what's coming:

Icon	Means	What to do with it
💡 Intuition	Plain-English idea + analogy	Read first. Never skip.
📌 Definition	Exact textbook definition	Memorise the wording.
📊 Derivation	Math, step by step	Re-derive with pen once.
📄 Worked Example	Real numbers, boxed answer	Cover answer, try yourself.
💻 Code	Runnable Python (NumPy/SciPy)	Run it; change a number.
🖼 Diagram	Labelled SVG figure	Redraw from memory.
⚙ Real-World	Where industry uses it	Fuels project ideas.
⚠ Exam Trap	Where students lose marks	Review night before.
★ Key Point	High-yield fact	Flashcard it.

Exam Blueprint

★ MARKS PATTERN

Total 100 = Internal Assessment (IA) **30** + University Exam (UE) **70**.

UE is a 2½–3 hr written paper. Expect: solve/derive questions (matrix ops, eigenvalues, Gram–Schmidt, SVD), short theory (define subspace, rank–nullity), and one application question.

🎯 CO → UNIT MAP

CO1 → Units 1, 4 (matrix algebra, linear systems)

CO2 → Unit 4 (row reduction, LU/QR, least squares)

CO3 → Units 2, 3 (linear maps, inner products)

CO4 → Unit 5 (digits, text mining)

Unit-wise weightage (inferred from lecture hours)

Unit	Title	Hours	Weight	Exam focus
1	Introduction to Vector Spaces	12	~25%	Subspace proofs, independence, basis, dimension, rank
2	Eigenvalues, Eigenvectors & Inner Product Spaces	10	~21%	Find eigen-pairs, diagonalize, norms, orthogonality
3	Linear Maps	12	~25%	Rank-nullity, matrix of a map, invertibility, isomorphism
4	Basic Matrix Methods for Applications	18	~37%	Row reduction, LU/QR/SVD, least squares, Gram–Schmidt
5	Mathematics Applied to Data Science	8	~17%	Digit recognition, LSI text mining, NMF

Unit 4 carries the most hours — budget your revision accordingly.

⚙️ TOOLBOX FOR THE WHOLE SUBJECT

Python `numpy`, `scipy.linalg`, `numpy.linalg`, `scikit-learn` (TruncatedSVD, NMF), `matplotlib`. Datasets: **MNIST** / **sklearn digits**, **20-Newsgroups**, **UCI** collections. Install: `pip install numpy scipy scikit-learn matplotlib`.

Table of Contents

Unit 1 — Introduction to Vector Spaces	12 hrs
Fields · Vector spaces · Subspaces · Sums & direct sums	
Span · Linear independence · Bases · Dimension · Rank · Change of basis	
Unit 2 — Eigenvalues, Eigenvectors & Inner Product Spaces	10 hrs
Eigen-pairs · Upper-triangular · Diagonalization · Inner products · Norms · Functionals	
Unit 3 — Linear Maps	12 hrs
$L(V,W)$ · Null space & range · Rank–nullity · Matrix of a map · Invertibility · Isomorphism · Quotients	
Unit 4 — Basic Matrix Methods for Applications	18 hrs
Linear systems · Row reduction · $Ax=b$ · Independence · Transformations	
Inverses · Factorizations · Least squares · SVD · Householder/QR · NMF · Gram–Schmidt	
Unit 5 — Mathematics Applied to Data Science	8 hrs
Handwritten digit recognition · SVD bases · Tangent distance · LSI · Clustering · NMF	
Appendix — Formula sheet · Glossary · Exam Q&A · Capstone	

★ THE ONE-SENTENCE SUMMARY OF THIS WHOLE SUBJECT

Data is a matrix. Every table, image, and document collection becomes a matrix; linear algebra gives you the tools to *compress it* (SVD, NMF), *solve with it* (least squares), *understand its structure* (rank, eigenvalues, subspaces), and *transform it* (linear maps). Master matrices and you have mastered the mathematical engine under machine learning.

🔗 NOTATION USED THROUGHOUT

$\mathbf{F}$ = a field ($\mathbb{R}$ or $\mathbb{C}$). $\mathbf{F}^n$ = column vectors of length n . Vectors $\mathbf{v}$ lowercase bold, matrices $\mathbf{A}$ uppercase. $\mathbf{A}^T$ transpose, $\mathbf{A}^{-1}$ inverse, $\|\mathbf{v}\|$ norm, $\langle \mathbf{u}, \mathbf{v} \rangle$ inner product, $\dim \mathbf{V}$ dimension, $\text{rank } \mathbf{A}$ rank, $\mathcal{L}(\mathbf{V}, \mathbf{W})$ set of linear maps $\mathbf{V} \rightarrow \mathbf{W}$, **null** $\mathbf{T}$ null space, **range** $\mathbf{T}$ range.

Introduction to Vector Spaces

Targets CO1, CO4
Axler Ch.1–2 · Lay Ch.4

💡 WHY START HERE

Before you can compress an image or solve a regression, you need a **space to work in**. A vector space is simply "a collection of things you can add together and scale, and the result stays inside the collection." Arrows in the plane, columns of numbers, polynomials, even greyscale images — all are vectors living in some vector space. This unit builds the vocabulary (subspace, span, basis, dimension, rank) that the entire subject is spoken in.

1.1 Fields — the numbers we scale with

💡 INTUITION

A **field** is any number system where you can **add, subtract, multiply and divide** (except by 0) and the ordinary arithmetic rules hold. The two we use: $\mathbb{R}$ (real numbers) and $\mathbb{C}$ (complex numbers). We write **F** to mean "either $\mathbb{R}$ or $\mathbb{C}$ " so we can prove things once for both.

🎯 DEFINITION (FIELD)

A set **F** with two operations $+$ and $\cdot$ is a *field* if both are commutative and associative, multiplication distributes over addition, there exist identities 0 (for $+$) and 1 (for $\cdot$) with $0 \neq 1$, every element has an additive inverse, and every **non-zero** element has a multiplicative inverse. **Examples:** $\mathbb{R}$, $\mathbb{C}$, $\mathbb{Q}$ are fields; $\mathbb{Z}$ (integers) is *not* — 2 has no integer reciprocal.

🎯 F^n , LISTS, $\mathbb{R}^n$ AND $\mathbb{C}^n$

A **list** of length n is an ordered tuple $(x_1, \dots, x_n)$. **F^n** is the set of all such lists with entries from F . So $\mathbb{R}^n$ = real column vectors of length n , $\mathbb{C}^n$ = complex ones. Addition is entry-wise; scalar multiplication multiplies every entry.

1.2 Vector Spaces

🎯 DEFINITION (VECTOR SPACE)

A **vector space over F** is a set V with an addition $V \times V \rightarrow V$ and a scalar multiplication $F \times V \rightarrow V$ satisfying the **eight axioms**: commutativity & associativity of $+$, an additive identity **0**, additive inverses, a multiplicative identity ($1 \cdot v = v$), associativity of scalar mult ($a(bv) = (ab)v$), and the two distributive laws $a(u+v) = au + av$ and $(a+b)v = av + bv$. Elements of V are **vectors**; elements of F are **scalars**.

Space	A "vector" is...	Zero vector	Where in DS
$\mathbb{R}^2, \mathbb{R}^3$	an arrow / point	origin (0,0)	2-D/3-D geometry, embeddings
$\mathbb{R}^n$	a column of n numbers	(0,...,0)	a data row / feature vector
$\mathbb{R}^{m \times n}$	an $m \times n$ matrix	zero matrix	a dataset, an image
$P(F)$	a polynomial	the zero polynomial	polynomial regression basis
F^∞	an infinite sequence	(0,0,...)	time-series / signals

Table 1.1 — Vector spaces you already know

TWO FACTS THAT FOLLOW FROM THE AXIOMS

(a) 0 is unique. Suppose 0 and 0' are both additive identities. Then $0 = 0 + 0' = 0'$. ■

(b) $0 \cdot v = 0$ for every v . Proof: $0 \cdot v = (0+0) \cdot v = 0 \cdot v + 0 \cdot v$. Add $-(0 \cdot v)$ to both sides $\Rightarrow 0 = 0 \cdot v$. ■ These tiny proofs are classic 2-mark exam questions — know them cold.

1.3 Subspaces

INTUITION

A **subspace** is a vector space living inside a bigger one — like a flat plane through the origin sitting inside 3-D space. It must contain the origin and be "closed": you can't add or scale your way out of it.

DEFINITION + SUBSPACE TEST

A subset $U \subseteq V$ is a **subspace** iff it satisfies three checks: **(1)** $0 \in U$ (contains zero); **(2)** $u, w \in U \Rightarrow u+w \in U$ (closed under addition); **(3)** $a \in F, u \in U \Rightarrow au \in U$ (closed under scalar multiplication).

WORKED EXAMPLE — IS IT A SUBSPACE?

Q: Is $U = \{(x, y, z) \in \mathbb{R}^3 : x + 2y - z = 0\}$ a subspace of $\mathbb{R}^3$?

(1) Zero: $0+0-0 = 0 \checkmark$ so $(0,0,0) \in U$.

(2) Addition: take $u=(x_1, y_1, z_1)$, $w=(x_2, y_2, z_2)$ both in U . Sum satisfies $(x_1+x_2)+2(y_1+y_2)-(z_1+z_2) = (x_1+2y_1-z_1) + (x_2+2y_2-z_2) = 0+0=0 \checkmark$

(3) Scaling: $a \cdot u$ gives $ax_1+2ay_1-az_1 = a(0)=0 \checkmark$

Answer: **Yes — U is a plane through the origin, a subspace.**

Contrast: $\{x+2y-z = 5\}$ is *not* a subspace — it misses the origin.

1.4 Sums & Direct Sums of Subspaces

DEFINITION

For subspaces $U_1, \dots, U_m$ of V , their **sum** is $U_1 + \dots + U_m = \{u_1 + \dots + u_m : u_i \in U_i\}$ — the smallest subspace containing all of them. The sum is a **direct sum**, written $U_1 \oplus \dots \oplus U_m$, when every vector in the sum has a **unique** representation as $u_1 + \dots + u_m$.

DIRECT-SUM CONDITION (TWO SUBSPACES)

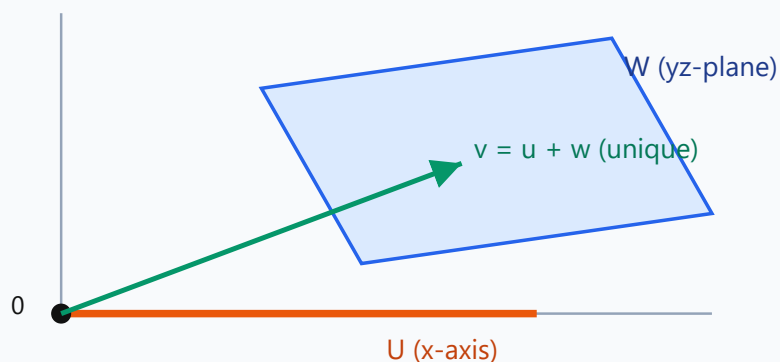
$U + W$ is direct $\Leftrightarrow U \cap W = \{0\}$.

Proof ($\Leftarrow$): suppose $U \cap W = \{0\}$ and $u+w = u'+w'$ with $u, u' \in U$, $w, w' \in W$. Then $u-u' = w'-w$ lies in **both** U and W , hence in $U \cap W = \{0\}$. So $u=u'$ and $w=w'$ — representation is unique. ■

WORKED EXAMPLE

In $\mathbb{R}^3$ let $U = x\text{-axis} = \{(x,0,0)\}$ and $W = yz\text{-plane} = \{(0,y,z)\}$. $U \cap W = \{(0,0,0)\} = \{0\}$, and $U+W = \mathbb{R}^3$ (any $(x,y,z) = (x,0,0) + (0,y,z)$). Therefore $\mathbb{R}^3 = U \oplus W$.

DIAGRAM — DIRECT SUM $\mathbb{R}^3 = \text{LINE} \oplus \text{PLANE}$



Every v splits uniquely into a piece on the line U and a piece in the plane W — that uniqueness is exactly what " $\oplus$ " guarantees.

EXAM TRAPS — UNIT 1A

- To disprove a subspace, the fastest kill is usually "**does it contain 0?**" — check that first.
- "Sum" $U+W$ is **not** the union $U \cup W$. The union is rarely a subspace.
- Direct sum needs intersection = $\{0\}$, not merely "small". Two distinct lines through the origin have intersection $\{0\}$ and give a direct sum (their plane).

1.5 Span & Linear Independence

💡 INTUITION

The **span** of a set of vectors is "everywhere you can reach" by adding scaled copies of them — the whole territory they generate. Vectors are **linearly independent** when none of them is redundant: you can't build any one from the others. Independent + spanning = a perfect, waste-free coordinate system (a basis).

🔗 DEFINITIONS

A **linear combination** of $v_1, \dots, v_m$ is $a_1v_1 + \dots + a_mv_m$ ($a_i \in F$).

span($v_1, \dots, v_m$) = the set of all such combinations. If it equals V , the list **spans** V .

The list is **linearly independent** if $a_1v_1 + \dots + a_mv_m = 0$ **forces** $a_1 = \dots = a_m = 0$. Otherwise it is **linearly dependent**.

📖 WORKED EXAMPLE — TEST INDEPENDENCE

Are $v_1 = (1, 2, 3)$, $v_2 = (2, 5, 7)$, $v_3 = (1, 3, 5)$ independent in $\mathbb{R}^3$? Solve $a_1v_1 + a_2v_2 + a_3v_3 = 0$. As a matrix, row-reduce the columns:

$$[[1, 2, 1], [2, 5, 3], [3, 7, 5]] \rightarrow R_2 - 2R_1, R_3 - 3R_1 \rightarrow [[1, 2, 1], [0, 1, 1], [0, 1, 2]] \rightarrow R_3 - R_2 \rightarrow [[1, 2, 1], [0, 1, 1], [0, 0, 1]]$$

Three pivots $\Rightarrow$ only solution is $a_1 = a_2 = a_3 = 0 \Rightarrow$ **independent.** (If the last row had become all zeros, they would be dependent.)

📖 LINEAR DEPENDENCE LEMMA (AXLER)

If $v_1, \dots, v_m$ is dependent, some v_j lies in $\text{span}(v_1, \dots, v_{j-1})$, and deleting it does **not** change the span.

Consequence: in a finite-dimensional space, the length of every *independent* list $\leq$ length of every *spanning* list. This single inequality is what makes "dimension" well-defined.

1.6 Bases & Coordinate Systems

🔗 DEFINITION (BASIS)

A **basis** of V is a list of vectors that is **both linearly independent and spanning**. Equivalently: a list B is a basis $\Leftrightarrow$ every $v \in V$ can be written as a linear combination of B in **exactly one way**. Those unique scalars are the **coordinates** of v w.r.t. B .

📖 WORKED EXAMPLE — COORDINATES

Standard basis of $\mathbb{R}^2$: $e_1 = (1, 0)$, $e_2 = (0, 1)$. Take basis $B = \{ b_1 = (1, 1), b_2 = (1, -1) \}$. Express $v = (4, 2)$ in B : solve

$c_1(1, 1) + c_2(1, -1) = (4, 2) \Rightarrow c_1 + c_2 = 4, c_1 - c_2 = 2 \Rightarrow c_1 = 3, c_2 = 1$. So **$[v]_B = (3, 1)$** . Same arrow, different "ruler."

1.7 Dimension

DEFINITION + KEY THEOREM

All bases of a finite-dimensional V have the **same length**; that common length is **dim V** . Examples: $\dim F^n = n$, $\dim \mathbb{R}^{m \times n} = mn$, $\dim P_k(F) = k+1$. **Useful facts:** in an n -dim space, any independent list of length n is automatically a basis, and any spanning list of length n is automatically a basis.

DIMENSION OF A SUM

$$\dim(U + W) = \dim U + \dim W - \dim(U \cap W)$$

Analogous to $|A \cup B| = |A| + |B| - |A \cap B|$ for sets. When the sum is direct, $\dim(U \cap W) = 0$ so $\dim(U \oplus W) = \dim U + \dim W$.

1.8 Rank

DEFINITION

The **rank** of a matrix A is the dimension of its column space (= dimension of its row space — they are always equal). It counts the number of **independent** directions / pivots. It is the single most important number describing a matrix: how much genuine information it carries.

WORKED EXAMPLE — RANK BY ROW REDUCTION

$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix}$. $R_2 - 2R_1 \rightarrow (0, 0, 0)$; $R_3 - R_1 \rightarrow (0, -1, -2)$. Two non-zero rows $\Rightarrow$ **rank $A = 2$** .
(Row 2 was just $2 \times$ row 1 — redundant.)

REAL-WORLD: RANK = INTRINSIC DIMENSIONALITY

A user-ratings matrix (users $\times$ movies) is huge but **low-rank**: a few hidden "taste factors" explain most ratings. Recommender systems (Netflix) exploit this by approximating the matrix with a low-rank factorization — the entire premise rests on rank being small.

CHANGE OF BASIS

Let B (old) and C (new) be two bases. The **change-of-basis matrix** P has the old basis vectors, expressed in C , as its columns. Then coordinates transform as

$$[v]_C = P \cdot [v]_B \quad \text{and back} \quad [v]_B = P^{-1}[v]_C$$

For a linear operator, the matrix changes as $A' = P A P^{-1}$ (a *similarity* transform) — this reappears in Unit 2 diagonalization.

WORKED EXAMPLE — CHANGE OF BASIS

Old = standard basis; new basis $C = \{ (1, 1), (1, -1) \}$. Columns of the matrix whose inverse maps standard $\rightarrow C$: $M = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $\det = -2$, $M^{-1} = (-1/2) \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 \\ 1/2 & -1/2 \end{bmatrix}$. For $v = (4, 2)$: $[v]_C = M^{-1}v = (1/2 \cdot 4 + 1/2 \cdot 2, 1/2 \cdot 4 - 1/2 \cdot 2) =$ **(3, 1)** — matches §1.6. $\checkmark$

```
import numpy as np
A = np.array([[1,2,1],[2,5,3],[3,7,5]], float) # columns = v1,v2,v3
print("rank =", np.linalg.matrix_rank(A))      # rank = 3 -> independent

# coordinates of v=(4,2) in basis B = {(1,1),(1,-1)}
B = np.array([[1,1],[1,-1]], float)
v = np.array([4,2], float)
coords = np.linalg.solve(B, v)
print("[v]_B =", coords)                      # [v]_B = [3. 1.]
```

Unit 1 Mini-Project

⚙️ MINI-PROJECT — "HOW MANY REAL DIMENSIONS IN THE IRIS DATA?"

Domain: botany / ML. **Dataset:** UCI Iris (150×4). **Task:** load the 4-feature matrix, compute its numerical rank, then mean-centre it and inspect how many singular values dominate (preview of SVD, Unit 4).

Deliverable: a short notebook arguing the "effective dimension" of Iris is ~2, and a 2-D scatter of the top two directions separating the species. Reinforces span, independence, rank on real data.

⚠️ EXAM TRAPS — UNIT 1B

- **Row rank = column rank** always. You may reduce either; the count matches.
- A basis needs **both** independence and spanning. A common error: giving an independent set that doesn't span (too few) or a spanning set that's dependent (too many).
- Change of basis: mind the direction. $[v]_C = P[v]_B$ where P's columns are the *old* vectors written in the *new* basis — easy to invert by mistake.
- $\dim(U+W)$ formula: don't forget to **subtract** $\dim(U \cap W)$.

Eigenvalues, Eigenvectors & Inner Product Spaces

Targets CO3, CO4

Axler Ch.5–6

💡 WHY THIS UNIT IS THE HEART OF DATA SCIENCE

An **eigenvector** of a matrix is a special direction that the matrix only **stretches**, never rotates; the stretch factor is its **eigenvalue**. These "natural axes" reveal a matrix's hidden structure — they power PCA (find directions of maximum variance), PageRank, vibration analysis, and stability of systems. **Inner products** then give us length and angle, letting us talk about *orthogonality* — the geometric backbone of least squares and SVD.

2.1 Eigenvalues & Eigenvectors

🔗 DEFINITION

For a square matrix (or operator) A , a scalar λ is an **eigenvalue** if there is a **non-zero** vector v with

$$A v = \lambda v$$

That v is an **eigenvector**. Geometrically: A sends v to a scalar multiple of itself — same line, scaled by λ .

📊 HOW TO FIND THEM — THE CHARACTERISTIC EQUATION

$Av = \lambda v \Leftrightarrow (A - \lambda I)v = 0$. A non-zero solution exists $\Leftrightarrow (A - \lambda I)$ is **singular** $\Leftrightarrow$

$$\det(A - \lambda I) = 0 \quad (\text{the characteristic polynomial})$$

Steps: (1) form $A - \lambda I$; (2) set its determinant to 0 and solve for λ ; (3) for each λ , solve $(A - \lambda I)v = 0$ to get the eigenvector(s). **Legend:** I = identity matrix, $\det$ = determinant, λ = eigenvalue, v = eigenvector.

📖 WORKED EXAMPLE — FULL EIGEN-DECOMPOSITION

$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.

1. Characteristic poly: $\det(\begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}) = (2-\lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda-1)(\lambda-3)$.

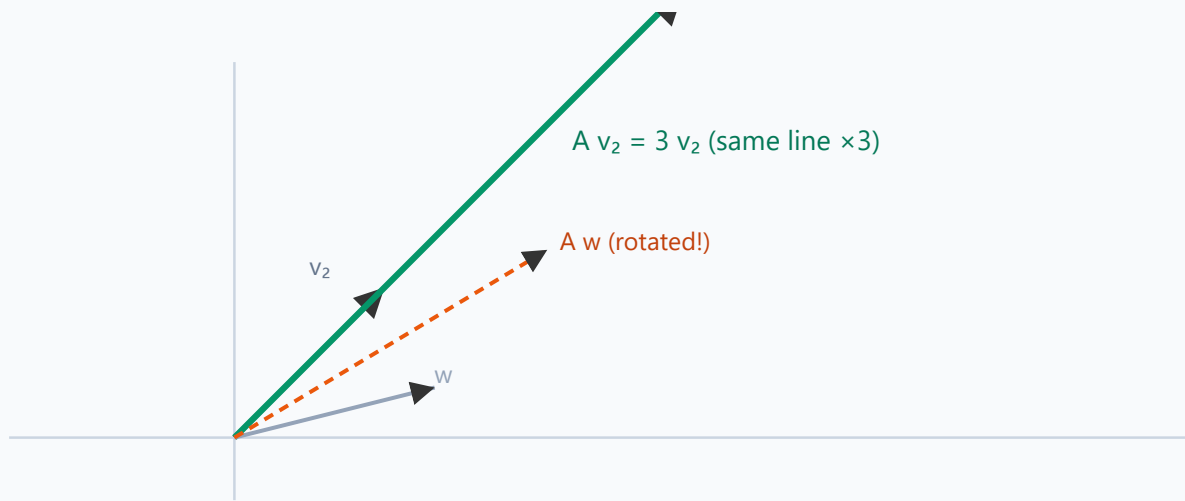
2. Eigenvalues: $\lambda_1 = 1, \lambda_2 = 3$.

3. For $\lambda_1 = 1$: $(A - I)v = 0 \rightarrow \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}v = 0 \rightarrow v_1 = (1, -1)$.

3. For $\lambda_2 = 3$: $(A - 3I)v = 0 \rightarrow \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}v = 0 \rightarrow v_2 = (1, 1)$.

$\lambda = 1$ (dir $(1, -1)$), $\lambda = 3$ (dir $(1, 1)$) — orthogonal, as expected for a symmetric A .

DIAGRAM — EIGENVECTORS KEEP THEIR DIRECTION



A stretches the eigenvector v_2 along its own line; an ordinary vector w gets rotated. That "no rotation" property is what makes eigenvectors special.

2.2 Eigenvectors & Upper-Triangular Matrices

★ KEY FACTS

- Eigenvectors for **distinct** eigenvalues are **linearly independent**.
- Over $\mathbb{C}$, every operator has an **upper-triangular matrix** with respect to some basis (Axler's Schur-type theorem).
- For a triangular matrix, the **eigenvalues are exactly the diagonal entries** — read them straight off.

📺 QUICK EXAMPLE

$T = [[4,7,2],[0,3,5],[0,0,9]]$ is upper-triangular $\Rightarrow$ eigenvalues are **4, 3, 9** — no work needed.

2.3 Eigenspaces & Diagonal Matrices

🔗 DEFINITION (EIGENSPACE & DIAGONALIZABILITY)

The **eigenspace** $E(\lambda, A) = \text{null}(A - \lambda I)$ = all eigenvectors for λ , plus 0. Its dimension is the **geometric multiplicity**. A is **diagonalizable** if V has a basis of eigenvectors — equivalently

$$A = P D P^{-1}, \quad D = \text{diag}(\lambda_1, \dots, \lambda_n), \quad \text{columns of } P = \text{eigenvectors}$$

Then $A^k = P D^k P^{-1}$ — powers become trivial (just power the diagonal).

📺 WORKED EXAMPLE — DIAGONALIZE $A = [[2,1],[1,2]]$

From §2.1: $P = [[1,1],[-1,1]]$ (eigenvectors as columns), $D = [[1,0],[0,3]]$. Check $A = P D P^{-1}$ with $P^{-1} = \frac{1}{2}[[1,-1],[1,1]]$:

$$A^{10} = P \cdot \text{diag}(1^{10}, 3^{10}) \cdot P^{-1} = P \cdot \text{diag}(1, 59049) \cdot P^{-1}$$

$A^{10} = \frac{1}{2}[[59050, 59048],[59048, 59050]]$ — computed instantly, no repeated multiplication.

```
import numpy as np
A = np.array([[2,1],[1,2]], float)
w, V = np.linalg.eig(A)          # w=eigenvalues, V=eigenvectors(columns)
print(w)                        # [1. 3.]
print(V)                        # [[-0.707  0.707],[0.707  0.707]]
# reconstruct: A == V @ diag(w) @ inv(V)
print(np.allclose(A, V @ np.diag(w) @ np.linalg.inv(V))) # True
```

⚙ REAL-WORLD: PCA IS AN EIGENPROBLEM

Principal Component Analysis diagonalizes the **covariance matrix**. Its eigenvectors are the principal directions; eigenvalues are the variance captured along each. Keeping the top-k eigenvectors compresses high-dimensional data (faces, gene expression) with minimal loss — the workhorse of dimensionality reduction.

2.4 Inner Products & Norms

💡 INTUITION

An **inner product** is a machine that eats two vectors and returns a number measuring how much they "agree" in direction. From it you get **length** (a vector's inner product with itself) and **angle** (via the cosine). It turns an abstract vector space into a geometry you can measure.

🔗 DEFINITION (INNER PRODUCT)

An **inner product** $\langle \cdot, \cdot \rangle$ on V over F assigns a scalar to each pair and satisfies: **(1) positivity** $\langle v, v \rangle \geq 0$ with equality only when $v=0$; **(2) linearity** in the first slot; **(3) conjugate symmetry** $\langle u, v \rangle = \text{conj}\langle v, u \rangle$ (just $\langle v, u \rangle$ over $\mathbb{R}$). The standard "dot product" on $\mathbb{R}^n$ is $\langle u, v \rangle = \sum u_i v_i = u^T v$.

🔗 NORM & ORTHOGONALITY

The **norm** (length) is $\|v\| = \sqrt{\langle v, v \rangle}$. Two vectors are **orthogonal** when $\langle u, v \rangle = 0$.

$$\|v\| = \sqrt{v_1^2 + \dots + v_n^2} \quad | \quad \cos \theta = \langle u, v \rangle / (\|u\| \|v\|)$$

📊 TWO INEQUALITIES YOU MUST KNOW

Cauchy-Schwarz: $|\langle u, v \rangle| \leq \|u\| \cdot \|v\|$ (equality $\Leftrightarrow$ parallel).

Triangle inequality: $\|u+v\| \leq \|u\| + \|v\|$.

Pythagoras: if $\langle u, v \rangle = 0$ then $\|u+v\|^2 = \|u\|^2 + \|v\|^2$.

Pythagoras proof: $\|u+v\|^2 = \langle u+v, u+v \rangle = \|u\|^2 + 2\langle u, v \rangle + \|v\|^2 = \|u\|^2 + \|v\|^2$ when $\langle u, v \rangle = 0$. ■

📌 WORKED EXAMPLE — LENGTH & ANGLE

$u=(3,4)$, $v=(4,0)$. $\|u\|=\sqrt{9+16}=5$, $\|v\|=4$, $\langle u, v \rangle = 3 \cdot 4 + 4 \cdot 0 = 12$. $\cos \theta = 12/(5 \cdot 4) = 0.6 \Rightarrow \theta = 53.13^\circ$. Check
Cauchy-Schwarz: $|12| \leq 5 \cdot 4 = 20$ ✓.

2.5 Linear Functionals on Inner-Product Spaces

🔗 DEFINITION + RIESZ REPRESENTATION

A **linear functional** $\varphi: V \rightarrow F$ is a linear map from V to its scalars (e.g. "sum the entries", "take the 3rd coordinate"). **Riesz Representation Theorem:** on a finite-dim inner-product space, every linear functional φ can be written as an inner product with a **unique fixed vector** u :

$$\varphi(v) = \langle v, u \rangle \quad \text{for all } v \in V$$

So "every measurement is secretly a dot product against some vector." This underlies the weight vector in linear models: a prediction $w^T x = \langle x, w \rangle$ is exactly a linear functional.

WORKED EXAMPLE — FIND THE RIESZ VECTOR

On $\mathbb{R}^3$ let $\varphi(x,y,z) = 2x - y + 5z$. Writing $\varphi(v) = \langle v, u \rangle = u_1x + u_2y + u_3z$, match coefficients:

$u = (2, -1, 5)$.

The functional "is" the dot product with u .

CODE — INNER PRODUCTS, NORMS, ANGLE

```
import numpy as np
u = np.array([3,4.]); v = np.array([4,0.])
dot = u @ v # 12.0
cos = dot/(np.linalg.norm(u)*np.linalg.norm(v))
print(np.degrees(np.arccos(cos))) # 53.13 degrees
print(u @ np.array([-4,3.])) # 0.0 -> orthogonal to (-4,3)
```

Concept	Answers the question	Core equation	DS use
Eigenvalue/vector	Which directions are only scaled?	$Av = \lambda v$	PCA, PageRank
Diagonalization	Can I make A "simple"?	$A = PDP^{-1}$	fast powers, dynamics
Inner product	How aligned are u and v ?	$\langle u, v \rangle = u^T v$	similarity, kernels
Norm	How long is v ?	$\ v\ = \sqrt{\langle v, v \rangle}$	regularization, distance
Orthogonality	Are they independent directions?	$\langle u, v \rangle = 0$	SVD, least squares

Table 2.1 — Eigen vs Inner-product ideas at a glance

Unit 2 Mini-Project

MINI-PROJECT — "EIGENFACES" ON FACES

Domain: computer vision / security. **Dataset:** Olivetti / LFW faces (sklearn). **Task:** stack face images as vectors, build the covariance matrix, extract top eigenvectors ("eigenfaces"), and reconstruct a face from just 20–50 of them. **Deliverable:** a figure showing the original vs low-rank reconstruction and a plot of variance captured per eigenvalue. Cements eigen-decomposition + norms as a compression tool.

EXAM TRAPS — UNIT 2

- Eigenvectors are **never zero** (0 always satisfies $Av = \lambda v$ trivially and is excluded).
- Not every matrix is diagonalizable: if geometric < algebraic multiplicity (a "defective" matrix like $\begin{bmatrix} 2 & 1 \\ 0 & 2 \end{bmatrix}$), it isn't.
- Symmetric real matrices are **always** diagonalizable with **orthogonal** eigenvectors — a favourite exam fact.
- Conjugate symmetry matters over $\mathbb{C}$: $\langle u, v \rangle$ and $\langle v, u \rangle$ are complex conjugates, not equal.
- Don't confuse eigenvalue (a number) with eigenvector (a direction).

💡 THE BIG IDEA

A **linear map** is a transformation that respects the vector-space structure: it takes lines to lines through the origin and preserves addition and scaling. Rotations, projections, derivatives, and — crucially — **every matrix** are linear maps. This unit reveals the deep truth of the subject: **matrices and linear maps are two faces of the same thing**. Once you see that, "solving $Ax=b$ " and "understanding a transformation" become the same activity.

3.1 Definition of Linear Maps

🔗 DEFINITION

A map $T: V \rightarrow W$ between vector spaces over the same field is **linear** if for all $u, v \in V$ and $a \in F$:

$$T(u + v) = T(u) + T(v) \text{ (additivity) and } T(av) = aT(v) \text{ (homogeneity)}$$

The set of all linear maps $V \rightarrow W$ is written $\mathcal{L}(V, W)$. A linear map from V to itself is an **operator**, $\mathcal{L}(V)$.

Consequence: every linear map sends 0 to 0. If $T(0) \neq 0$, it's not linear.

📖 WORKED EXAMPLE — LINEAR OR NOT?

- (a) $T(x, y) = (2x + y, x - y)$: check $T(u + v) = T(u) + T(v)$ and $T(av) = aT(v)$ — both hold $\Rightarrow$ **linear**.
 (b) $T(x, y) = (x + 1, y)$: $T(0, 0) = (1, 0) \neq (0, 0) \Rightarrow$ **not linear** (it's *affine*).
 (c) $T(x, y) = (x^2, y)$: $T(2 \cdot (1, 1)) = (4, 2)$ but $2 \cdot T(1, 1) = (2, 2) \Rightarrow$ **not linear**.

3.2 Algebraic Operations on $\mathcal{L}(V, W)$

🔗 DEFINITION

Linear maps can be **added** $(S + T)(v) = S(v) + T(v)$, **scaled** $(aT)(v) = a \cdot T(v)$, and **composed** $(ST)(v) = S(T(v))$. Sums and scalar multiples of linear maps are linear, so $\mathcal{L}(V, W)$ is itself a vector space. Composition corresponds exactly to **matrix multiplication** — which is *why* matrix multiplication is defined the "strange" row-times-column way.

3.3 Null Space & Injectivity

🔗 DEFINITION (NULL SPACE / KERNEL)

The **null space** (kernel) of T is

$$\text{null } T = \{ v \in V : T(v) = 0 \}$$

It is a subspace of V . **Key theorem:** T is **injective** (one-to-one) $\Leftrightarrow$ **null $T = \{0\}$** . Intuition: the only vector crushed to zero is zero itself, so no two inputs collide.

3.4 Range & Surjectivity

🔗 DEFINITION (RANGE / IMAGE)

The **range** (image) of T is $\text{range } T = \{ T(v) : v \in V \}$, a subspace of W . T is **surjective** (onto) $\Leftrightarrow \text{range } T = W$.
For a matrix A , $\text{range} = \text{column space}$, $\text{null} = \text{solution set of } Ax=0$.

3.5 Fundamental Theorem of Linear Maps (Rank–Nullity)

📊 THE THEOREM — THE MOST USED RESULT IN THE SUBJECT

For $T: V \rightarrow W$ with V finite-dimensional,

$$\dim V = \dim(\text{null } T) + \dim(\text{range } T)$$

In matrix language, for an $m \times n$ matrix A : **$n = \text{nullity}(A) + \text{rank}(A)$** . **nullity** = dim of null space (free variables), **rank** = dim of column space (pivots). Every input dimension is either "killed" (nullity) or "survives into the output" (rank).

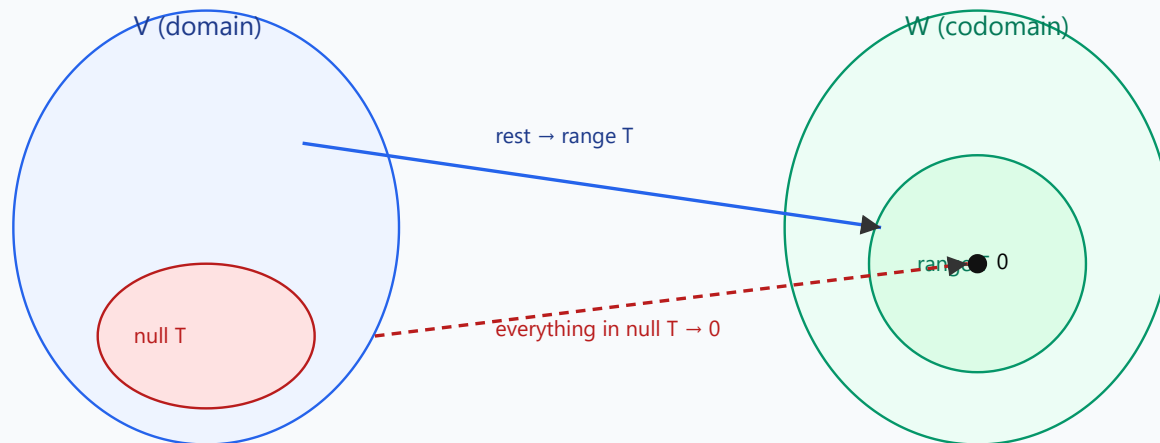
📌 WORKED EXAMPLE — RANK–NULLITY IN ACTION

$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{bmatrix}$ (2×3 , so $n=3$). Row reduce: $R_2 - 2R_1 \rightarrow (0,0,0)$. One pivot $\Rightarrow \text{rank} = 1$. Then $\text{nullity} = n - \text{rank} = 3 - 1 = 2$.

Verify null space: $x_1 + 2x_2 + 3x_3 = 0 \Rightarrow x_1 = -2x_2 - 3x_3$, two free variables $(x_2, x_3) \Rightarrow \dim = 2 \checkmark$.

$$\text{rank } 1 + \text{nullity } 2 = 3 = n \checkmark$$

📊 DIAGRAM — A LINEAR MAP SPLITS ITS DOMAIN



$\dim(\text{null } T)$ directions collapse to 0; the remaining $\dim(\text{range } T)$ directions map onto the image. Their sizes always sum to $\dim V$.

★ COROLLARIES YOU CAN QUOTE

- If $\dim V > \dim W$, T **cannot** be injective (something must collapse).
- If $\dim V < \dim W$, T **cannot** be surjective (can't fill a bigger space).
- For an operator on a finite-dim space ($V=W$): **injective** $\Leftrightarrow$ **surjective** $\Leftrightarrow$ **invertible**.

3.6 Representing a Linear Map by a Matrix

THE BRIDGE MAP $\leftrightarrow$ MATRIX

Pick a basis $(v_1, \dots, v_n)$ of V and $(w_1, \dots, w_m)$ of W . The **matrix of T** , written $\mathcal{M}(T)$, has as its **j -th column the coordinates of $T(v_j)$** in the W -basis. Then applying T becomes multiplying by that matrix:

$$[T(v)]_W = \mathcal{M}(T) \cdot [v]_V$$

"To transform a vector, transform each basis vector and record where it goes."

WORKED EXAMPLE — BUILD THE MATRIX

$T: \mathbb{R}^2 \rightarrow \mathbb{R}^2$, rotation by 90° counter-clockwise. It sends $e_1=(1,0) \mapsto (0,1)$ and $e_2=(0,1) \mapsto (-1,0)$. Put those images as columns:

$$\mathcal{M}(T) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Check on $v=(2,3)$: $\mathcal{M}(T)v = (0 \cdot 2 - 1 \cdot 3, 1 \cdot 2 + 0 \cdot 3) = \boxed{(-3, 2)}$ — a 90° turn. $\checkmark$

3.7 Invertible Linear Maps & Isomorphisms

DEFINITION

$T \in \mathcal{L}(V, W)$ is **invertible** if there is $S \in \mathcal{L}(W, V)$ with $ST = I_V$ and $TS = I_W$. T is invertible $\Leftrightarrow$ it is **injective and surjective** (bijective). When such a T exists, V and W are **isomorphic** ($V \cong W$) — "the same space wearing different labels."

★ THE CLASSIFICATION THEOREM

Two finite-dimensional spaces over F are isomorphic $\Leftrightarrow$ **they have the same dimension**. Consequence: **every** n -dimensional space over F is isomorphic to F^n . This is *why* we can always reduce abstract problems to concrete column vectors and matrices.

3.8 Linear Map as Matrix Multiplication · Operators

KEY CORRESPONDENCE

Under fixed bases: linear maps $\leftrightarrow$ matrices, **composition** $\leftrightarrow$ **matrix product**, invertible map $\leftrightarrow$ invertible matrix. An **operator** ($T: V \rightarrow V$) gets a **square** matrix; changing basis conjugates it, $A' = P A P^{-1}$ (the similarity transform from Unit 1) — which is exactly diagonalization when the new basis is eigenvectors.

3.9 Products & Quotients of Vector Spaces

PRODUCT SPACE

The **product** $V_1 \times \dots \times V_m$ is the set of tuples $(v_1, \dots, v_m)$ with componentwise operations. **$\dim(V_1 \times \dots \times V_m) = \dim V_1 + \dots + \dim V_m$** . There is a natural link: the map $(u, w) \mapsto u + w$ shows a direct sum $U \oplus W$ is isomorphic to the product $U \times W$.

QUOTIENT SPACE

Given a subspace $U \subseteq V$, the **quotient** V/U is the set of **affine cosets** $v+U = \{v+u : u \in U\}$, with $(v+U)+(w+U) = (v+w)+U$ and $a(v+U) = (av)+U$. Intuitively you "collapse U to a point" and keep only the directions transverse to U .

$$\dim(V/U) = \dim V - \dim U \quad (\text{the } \mathbf{\text{codimension}} \text{ of } U)$$

WORKED EXAMPLE — A QUOTIENT YOU CAN PICTURE

$V = \mathbb{R}^2$, $U = x\text{-axis}$. Each coset $(a,b)+U$ is the **horizontal line at height b** — all points with the same y . So $\mathbb{R}^2/U$ is parameterised by the single number b : **$\mathbb{R}^2/U \cong \mathbb{R}$, $\dim = 2 - 1 = 1$** . The quotient "forgets x , remembers height."

FIRST ISOMORPHISM THEOREM (TIES IT TOGETHER)

For any linear $T: V \rightarrow W$,

$$V / \text{null } T \cong \text{range } T$$

Taking dimensions recovers rank–nullity: $\dim V - \dim(\text{null } T) = \dim(\text{range } T)$. The quotient literally "quotients out" exactly what T destroys.

CODE — MATRIX OF A MAP, NULL SPACE, RANK–NULLITY

```
import numpy as np
from scipy.linalg import null_space
A = np.array([[1,2,3],[2,4,6]], float)      # a Linear map  $\mathbb{R}^3 \rightarrow \mathbb{R}^2$ 
r = np.linalg.matrix_rank(A)                # rank = 1
N = null_space(A)                           # basis of null  $T$  (3x2)
print("rank", r, "nullity", N.shape[1])     # rank 1 nullity 2
print(r + N.shape[1] == A.shape[1])        # True (n = rank + nullity)

# 90-degree rotation operator
R = np.array([[0,-1],[1,0]], float)
print(R @ np.array([2,3]))                 # [-3.  2.]
```

Linear map T	Matrix A	Meaning
null T	solutions of $Ax=0$	what collapses to 0
range T	column space of A	reachable outputs
injective	null = $\{0\}$, full column rank	no info lost
surjective	rank = m (rows)	fills the codomain
invertible	square, $\det \neq 0$	bijection, A^{-1} exists
composition ST	product S·A	do T then S

Table 3.1 — Map language $\leftrightarrow$ Matrix language (memorise this)

Unit 3 Mini-Project

MINI-PROJECT — IMAGE TRANSFORMATIONS AS LINEAR MAPS

Domain: graphics / augmentation. **Dataset:** any image or **sklearn digits**. **Task:** implement rotation, scaling, shear, and reflection as 2×2 matrices; apply them to the pixel coordinate grid; verify composition = matrix product (rotate-then-shear vs the single product matrix). **Deliverable:** a panel of transformed images with the matrix printed under each — a visual proof that "linear map = matrix." Great intuition for data-augmentation pipelines in deep learning.

EXAM TRAPS — UNIT 3

- Every linear map sends $0 \rightarrow 0$. Any map with a "+constant" (affine) is **not** linear.
- Rank-nullity uses dim of the **domain** (columns n), not the codomain m . Classic slip.
- Matrix of T : columns are **images of basis vectors**, in the target basis — don't transpose by accident.
- Isomorphic $\Leftrightarrow$ equal dimension — but only for **finite-dimensional** spaces over the same field.
- Coset $v+U = v'+U \Leftrightarrow v-v' \in U$. Different-looking representatives can be the same point of V/U .

Basic Matrix Methods for Applications

Targets CO1, CO2
Lay Ch.1–2,6,7 · Eldén

THE WORKHORSE UNIT

This is where linear algebra becomes **computation**. You'll solve systems $Ax=b$, invert and factor matrices, and — most importantly for data science — handle the case where there is **no exact solution** (more equations than unknowns) using **least squares** and the crown jewel, the **SVD**. Nearly every exam's long-answer questions live here, so we go slow and solve everything numerically.

4.1 Systems of Linear Equations

DEFINITION

A **system** of m equations in n unknowns is written $Ax = b$, where A is the $m \times n$ coefficient matrix, x the unknowns, b the right-hand side. It has either **no solution** (inconsistent), **exactly one**, or **infinitely many**. The **augmented matrix** $[A \mid b]$ packages everything.

4.2 Row Reduction & Echelon Forms

DEFINITION (REF & RREF)

Row Echelon Form (REF): all-zero rows at bottom; each leading entry (pivot) is to the right of the one above. **Reduced REF (RREF)**: additionally every pivot = 1 and is the only non-zero in its column. We reach them with three **elementary row operations**: swap rows, scale a row, add a multiple of one row to another. Pivot columns $\Rightarrow$ basic variables; non-pivot $\Rightarrow$ free variables.

WORKED EXAMPLE — GAUSSIAN ELIMINATION

Solve $x + 2y + z = 8$, $2x + 3y = 5$, $4y + z = 11$.

Augmented: $[[1,2,1|8],[2,3,0|5],[0,4,1|11]]$.

R2 $\leftarrow$ R2 - 2R1: $(0, -1, -2 \mid -11)$. **R3 $\leftarrow$ R3 + 4R2**: $(0, 0, -7 \mid -33)$.

Back-substitute: $z = 33/7 \approx 4.714$; $-y - 2z = -11 \Rightarrow y = 11 - 2z = 11 - 9.428 = 1.571$; $x = 8 - 2y - z =$

$8 - 3.143 - 4.714 = 0.143$. **$(x,y,z) \approx (0.143, 1.571, 4.714)$**

4.3 Vector & Matrix Equations; the Solution Set

THREE VIEWS OF THE SAME THING

(1) Matrix eq: $Ax = b$. **(2) Vector eq**: $x_1a_1 + \dots + x_na_n = b$ (b as a combination of A 's columns). **(3) System of scalar equations**. So **$Ax=b$ is solvable $\Leftrightarrow b$ lies in the column space of A** .

Solution-set structure: if x_p is any particular solution, the full set is

$$x = x_p + x_h, \text{ where } x_h \in \text{null}(A) \text{ (solves } Ax=0\text{)}$$

"one particular answer + all the ways to shift without leaving the solution."

4.4 Applications of Linear Systems

WHERE $AX=B$ SHOWS UP

Network/traffic flow (conservation at each node), **electrical circuits** (Kirchhoff's laws), **chemical equation balancing**, **economic input–output (Leontief) models**, and **polynomial curve fitting**. In ML, the **normal equations** of linear regression are a linear system (§4.9).

4.5 Linear Independence & Linear Transformations (matrix view)

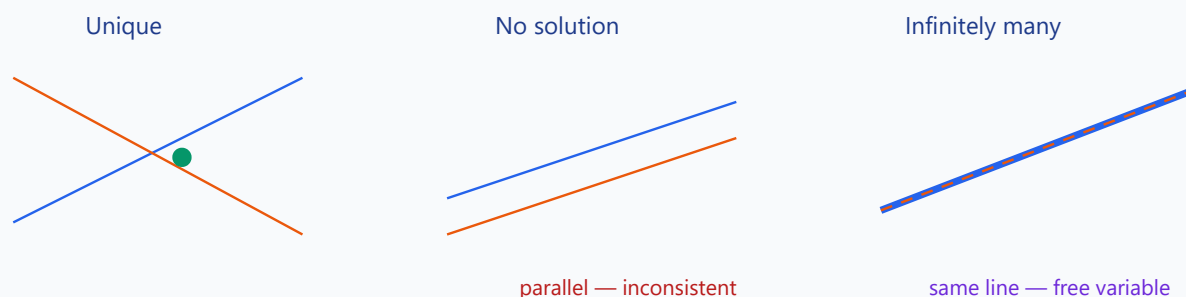
RECAP IN MATRIX FORM

Columns of A are independent $\Leftrightarrow Ax=0$ has **only** $x=0 \Leftrightarrow$ every column is a pivot column $\Leftrightarrow \text{rank } A = n$ (number of columns). Every linear transformation $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ is $x \mapsto Ax$ for the **standard matrix** $A = [T(e_1) \dots T(e_n)]$ (its columns are images of the standard basis — same recipe as Unit 3).

WORKED EXAMPLE — STANDARD MATRIX OF A PROJECTION

Projection onto the x -axis: $T(x,y)=(x,0)$. Then $T(e_1)=(1,0)$, $T(e_2)=(0,0)$, so **$A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$** . Note rank = 1, nullity = 1 (the whole y -axis collapses).

DIAGRAM — THE THREE OUTCOMES OF $AX=B$



Two lines meeting = unique; parallel = none; coincident = infinitely many. Higher dimensions generalise this exactly.

CODE — SOLVE, AND DETECT THE THREE CASES

```
import numpy as np
A = np.array([[1,2,1],[2,3,0],[0,4,1]], float)
b = np.array([8,5,11], float)
x = np.linalg.solve(A, b)          # unique: [0.143 1.571 4.714]
print(x)
# consistency check for general/rectangular A: compare ranks
Aug = np.column_stack([A, b])
print(np.linalg.matrix_rank(A), np.linalg.matrix_rank(Aug)) # 3 3 -> consistent
```

4.6 Matrix Norms & The Inverse

MATRIX NORM

A **matrix norm** measures a matrix's "size" / amplification. The **Frobenius norm** is $\|A\|_F = \sqrt{\sum a_{ij}^2}$; the **spectral (2-)norm** $\|A\|_2 = \text{largest singular value } \sigma_1$ (Unit 4.10). The **condition number** $\kappa(A) = \sigma_1/\sigma_n$ tells you how sensitive $Ax=b$ is to errors — huge $\kappa \Rightarrow$ ill-conditioned.

INVERSE OF A MATRIX

A square A is **invertible** if some A^{-1} satisfies $AA^{-1}=A^{-1}A=I$. For 2×2 :

$$[[a,b],[c,d]]^{-1} = 1/(ad-bc) \cdot [[d,-b],[-c,a]], \text{ provided } \det = ad-bc \neq 0$$

Then $Ax=b$ has the unique solution $x = A^{-1}b$. (In practice we **never** invert — we factor and solve; see §4.8.)

WORKED EXAMPLE — 2×2 INVERSE

$A = [[4,3],[6,3]]$, $\det = 4 \cdot 3 - 3 \cdot 6 = -6$. $A^{-1} = (-1/6)[[3,-3],[-6,4]] =$ $[[-0.5, 0.5], [1, -0.667]]$. Check $A \cdot A^{-1} = I$. ✓

4.7 Characterizations of Invertible Matrices (IMT)

★ INVERTIBLE MATRIX THEOREM — ALL EQUIVALENT FOR SQUARE A

A invertible $\Leftrightarrow \det A \neq 0 \Leftrightarrow \text{rank } A = n \Leftrightarrow \text{columns independent} \Leftrightarrow \text{columns span } \mathbb{R}^n \Leftrightarrow Ax=0 \text{ has only } x=0 \Leftrightarrow Ax=b \text{ has a unique solution for every } b \Leftrightarrow A \text{ is row-equivalent to } I \Leftrightarrow 0 \text{ is not an eigenvalue} \Leftrightarrow A^T \text{ is invertible. Any one fails} \Rightarrow \text{all fail. A staple exam question.}$

4.8 Partitioned Matrices & Matrix Factorizations (LU)

BLOCK / PARTITIONED MATRICES

Splitting a matrix into **blocks** lets you multiply block-wise (as if blocks were scalars, order kept). Used for efficiency and for deriving factorizations.

LU FACTORIZATION

Gaussian elimination = writing $A = LU$, L lower-triangular (the multipliers), U upper-triangular (the echelon form). Then $Ax=b$ is solved in two easy triangular sweeps:

Solve $Ly = b$ (forward), then $Ux = y$ (backward)

Reusable for many b 's — the reason LU beats computing A^{-1} .

WORKED EXAMPLE — LU

$A = \begin{bmatrix} 2 & 4 \\ 6 & 3 \end{bmatrix}$. Multiplier $m_{21} = 6/2 = 3$; $R_2 - 3R_1 \rightarrow (0, -9)$. So $L = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$, $U = \begin{bmatrix} 2 & 4 \\ 0 & -9 \end{bmatrix}$.
(check: $LU = \begin{bmatrix} 2 & 4 \\ 6 & 12 - 9 = 3 \end{bmatrix}$).

A = LU ✓

4.9 Subspaces of $\mathbb{R}^n$, Dimension & Rank; The Least-Squares Problem

FOUR FUNDAMENTAL SUBSPACES

For an $m \times n$ matrix A : **column space** $\text{Col } A \subseteq \mathbb{R}^m$ (dim = rank), **null space** $\text{Nul } A \subseteq \mathbb{R}^n$ (dim = nullity), **row space** $\text{Row } A$ (dim = rank), **left null space** $\text{Nul } A^T$. $\text{Col } A \perp \text{Nul } A^T$ and $\text{Row } A \perp \text{Nul } A$ — orthogonal complements.

LEAST SQUARES — WHEN $AX=B$ HAS NO SOLUTION

With more equations than unknowns ($m > n$), b usually isn't in $\text{Col } A$. Best we can do: minimise the error $\|Ax - b\|^2$. The minimiser $\hat{x}$ makes the residual **orthogonal to Col A**, giving the **normal equations**:

$$A^T A \hat{x} = A^T b \Rightarrow \hat{x} = (A^T A)^{-1} A^T b$$

This **is** linear regression. Geometrically $A\hat{x}$ is the **orthogonal projection** of b onto $\text{Col } A$.

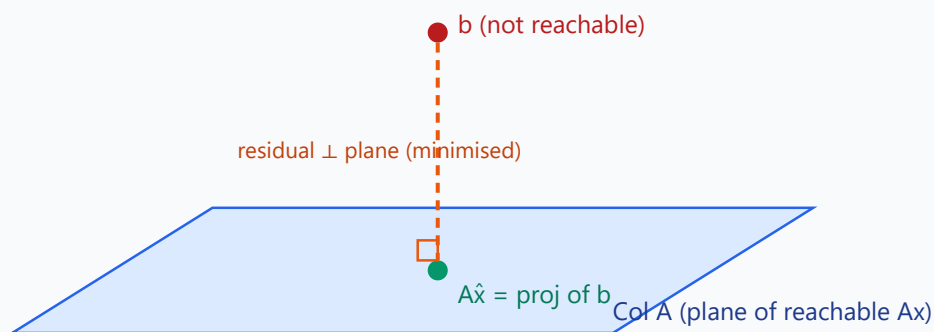
WORKED EXAMPLE — FIT A LINE $Y = MX + C$

Points $(1,1), (2,2), (3,2)$. Model: $A = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$ (cols: $x, 1$), $b = (1, 2, 2)$, unknown (m, c) .

$A^T A = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$, $A^T b = \begin{bmatrix} 11 \\ 5 \end{bmatrix}$. Solve: $\det = 14 \cdot 3 - 36 = 6$. $(A^T A)^{-1} = (1/6) \begin{bmatrix} 3 & -6 \\ -6 & 14 \end{bmatrix}$. $\hat{x} = (1/6) \begin{bmatrix} 3 & -6 \\ -6 & 14 \end{bmatrix} \cdot \begin{bmatrix} 11 \\ 5 \end{bmatrix} = (1/6) \begin{bmatrix} 33 - 30 \\ -66 + 70 \end{bmatrix} = (1/6) \begin{bmatrix} 3 \\ 4 \end{bmatrix}$.

$m = 0.5, c \approx 0.667 \rightarrow \hat{y} = 0.5x + 0.667$.

DIAGRAM — LEAST SQUARES = PROJECTION ONTO COL A



The best-fit $\hat{x}$ drops b straight down onto the plane of achievable outputs; the leftover (residual) is perpendicular — that's the whole content of the normal equations.

CODE — LEAST SQUARES THREE WAYS

```
import numpy as np
A = np.array([[1,1],[2,1],[3,1]], float); b = np.array([1,2,2])
xhat,_,_ = np.linalg.lstsq(A, b, rcond=None) # [0.5 0.667] (m, c)
print(xhat)
# normal equations by hand
print(np.linalg.solve(A.T@A, A.T@b)) # same
```


4.10 Singular Value Decomposition (SVD)

💡 INTUITION — THE "MASTER FACTORIZATION"

Every matrix, even non-square, can be broken into **rotate** → **stretch** → **rotate**. The stretch factors (singular values) rank the directions from most to least important. Keep the top few and you get the **best low-rank approximation** of your data — the mathematical core of compression, PCA, recommender systems, and noise removal.

🔗 DEFINITION

For any $m \times n$ matrix A of rank r :

$$A = U \Sigma V^T$$

U ($m \times m$) and V ($n \times n$) are **orthogonal** (columns orthonormal); Σ ($m \times n$) is diagonal with **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Columns of U = **left singular vectors** (eigenvectors of AA^T), columns of V = **right singular vectors** (eigenvectors of $A^T A$), and $\sigma_i = \sqrt{\text{eigenvalue of } A^T A}$.

📊 ECKART-YOUNG — BEST LOW-RANK APPROXIMATION

Keeping the largest k singular triplets gives

$$A_k = \sum_{i=1..k} \sigma_i u_i v_i^T$$

which is the **closest rank- k matrix to A** (in both Frobenius and spectral norm). Error = σ_{k+1} . This one fact is why SVD is everywhere in data science.

📌 WORKED EXAMPLE — SVD VIA $A^T A$

$A = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$ (already diagonal-ish). $A^T A = \begin{bmatrix} 9 & 0 \\ 0 & 4 \end{bmatrix}$, eigenvalues 9, 4 $\Rightarrow \sigma_1=3, \sigma_2=2$. So $\Sigma = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$, with sign folded into U/V . **Singular values 3 and 2** (note: singular values are always ≥ 0 , unlike the eigenvalue -2).

💻 CODE — SVD & RANK-K IMAGE COMPRESSION

```
import numpy as np
A = np.random.rand(6,4)
U,s,Vt = np.linalg.svd(A, full_matrices=False)
print(s)                                # singular values, descending
k = 2
Ak = U[:, :k] * s[:k] @ Vt[:k, :]      # best rank-2 approximation
print(np.linalg.norm(A-Ak))             # == s[k] (Eckart-Young)
```

4.11 Householder Transformation & QR Decomposition

QR & HOUSEHOLDER

QR: $A = QR$, Q orthogonal, R upper-triangular. It solves least squares stably: $R\hat{x} = Q^Tb$ (one back-substitution, no A^TA needed — better conditioning). A **Householder reflection** $H = I - 2vv^T/(v^Tv)$ is an orthogonal matrix that reflects a vector to zero out entries below a pivot; applying a sequence of them turns A into R (and their product is Q). More numerically stable than Gram–Schmidt.

WORKED EXAMPLE — QR BY NUMPY

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \rightarrow Q \text{ (3} \times 2 \text{ orthonormal)}, R = \begin{bmatrix} -1.414 & -0.707 \\ 0 & -1.225 \end{bmatrix}$$

Columns of Q are orthonormal ($Q^TQ = I$); R is upper-triangular. Least squares: solve $R\hat{x} = Q^Tb$.

4.12 Non-Negative Matrix Factorization (NMF) & Bidiagonalization

NMF

Approximate a **non-negative** matrix $A \approx WH$ with W, H **also non-negative** (entries ≥ 0). Because nothing can cancel, the factors are **parts-based and interpretable** — W holds "topics/parts", H their weights. Ideal for text (topics), images (facial parts), and audio.

Bidiagonalization (Golub–Kahan / LGK): reduce A to a bidiagonal matrix using orthogonal transforms — the first step inside SVD algorithms and used in large sparse problems (Unit 5's LGK).

4.13 Inner Product, Length, Orthogonality · Orthogonal Sets & Projections · Gram–Schmidt

ORTHOGONAL / ORTHONORMAL SETS & PROJECTION

A set is **orthogonal** if every pair has dot product 0; **orthonormal** if also each has length 1. Orthogonal sets are automatically independent. The **orthogonal projection** of y onto a subspace with orthogonal basis $\{u_1, \dots, u_p\}$:

$$\hat{y} = \sum (\langle y, u_j \rangle / \langle u_j, u_j \rangle) u_j$$

GRAM–SCHMIDT PROCESS

Turns any independent basis $\{x_1, \dots, x_p\}$ into an orthogonal one $\{v_1, \dots, v_p\}$:

$$v_1 = x_1; \quad v_k = x_k - \sum_{j < k} (\langle x_k, v_j \rangle / \langle v_j, v_j \rangle) v_j$$

Normalise each v_k to get an orthonormal basis (the Q of QR).

WORKED EXAMPLE — GRAM–SCHMIDT

$x_1 = (1, 1, 0)$, $x_2 = (1, 0, 1)$. $v_1 = (1, 1, 0)$. Projection coeff = $\langle x_2, v_1 \rangle / \langle v_1, v_1 \rangle = 1/2$. $v_2 = (1, 0, 1) - 1/2(1, 1, 0) = (1/2, -1/2, 1)$.

Check $\langle v_1, v_2 \rangle = 1/2 - 1/2 + 0 = 0 \checkmark$.

Orthogonal basis: $(1, 1, 0)$, $(1/2, -1/2, 1)$.

```
import numpy as np
X = np.array([[1,1],[1,0],[0,1]], float)
Q,R = np.linalg.qr(X)           # orthonormal Q (Gram-Schmidt result)
print(np.round(Q.T@Q,6))        # identity -> orthonormal columns

from sklearn.decomposition import NMF
V = np.abs(np.random.rand(6,4))
W = NMF(n_components=2, init='nndsvda', max_iter=500).fit_transform(V)
print(W.shape)                  # (6, 2) non-negative parts
```

Factorization	Form	Best for
LU	$A = LU$	solving square $Ax=b$, many b's
QR	$A = QR$	stable least squares, Gram-Schmidt
SVD	$A = U\Sigma V^T$	rank, compression, PCA, pseudo-inverse
NMF	$A \approx WH$ (≥ 0)	interpretable topics/parts
Eigen	$A = PDP^{-1}$	square, dynamics, powers

Table 4.1 — Choose the right factorization

Unit 4 Mini-Project

MINI-PROJECT — PREDICT HOUSE PRICES WITH LEAST SQUARES

Domain: real estate / finance. **Dataset:** **Kaggle House Prices** or sklearn **California Housing**. **Task:** build A (features + intercept column), solve the normal equations *and* via QR, compare coefficients and residuals; then use SVD to inspect the condition number and detect multicollinearity. **Deliverable:** a report showing your hand-rolled least squares matches sklearn's `LinearRegression` and an SVD scree plot of feature importance.

EXAM TRAPS — UNIT 4

- Normal equations need A of **full column rank**; if columns are collinear, $A^T A$ is singular — use SVD/pseudo-inverse.
- Singular values are **always ≥ 0** and are $\sigma_i = \sqrt{\text{eigenvalues of } A^T A}$ — not the eigenvalues of A .
- In $A=U\Sigma V^T$ don't forget the **transpose** on V . Getting U and V swapped loses marks.
- Prefer QR/LU over computing A^{-1} explicitly — inverting is slower and less stable (mention this for method marks).
- Gram-Schmidt: subtract projections onto the **already-orthogonalised** v_j , not the original x_j .
- NMF requires the input to be **non-negative**; it won't run on centred data with negatives.

Mathematics Applied to Data Science

Targets CO4
Eldén, Matrix Methods in Data Mining

BRINGING IT ALL HOME

Everything from Units 1–4 now solves **two real problems**: (1) **recognising handwritten digits** from images, and (2) **searching & organising text**. Both reduce to the same trick — turn the data into a matrix and use SVD/NMF to expose structure. This unit follows **Eldén's** classic treatment.

5.1 Handwritten Digit Recognition — a simple algorithm

SETUP

Each 16×16 (or 28×28) digit image is flattened into a **vector** (256 or 784 numbers). A training set of many labelled digits becomes a big matrix. The **simplest classifier**: compute the **mean image** ("centroid") of each digit class 0–9; classify a new image by the **nearest centroid** (smallest Euclidean distance).

NEAREST-CENTROID RULE

$$\text{predict}(x) = \operatorname{argmin}_{d \in \{0..9\}} \|x - \mu_d\| \quad \text{where } \mu_d = \text{mean of class-}d \text{ images}$$

Fast and surprisingly decent — but blind to the many *styles* of writing each digit. That's what SVD fixes.

5.2 Classification using SVD Bases

THE KEY UPGRADE

Instead of one average image per digit, model each digit class by a small **subspace** that captures its many writing styles. Take all training images of digit d , stack them as columns of a matrix A_d , and compute its **SVD**. The first few left singular vectors (u_i) form an **orthonormal basis** — the dominant "shapes" of that digit.

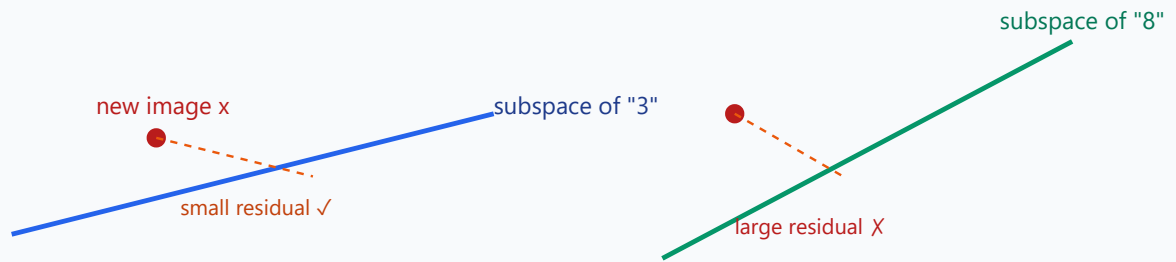
SVD-BASIS CLASSIFIER

For each class d , keep the top- k left singular vectors $U_d = [u_1 \dots u_k]$. To classify a new image x , measure how well each class subspace explains it — the **residual** of projecting x onto U_d :

$$r_d(x) = \|x - U_d U_d^T x\|; \quad \text{predict} = \operatorname{argmin}_d r_d(x)$$

Small residual $\Rightarrow x$ lies close to that digit's subspace $\Rightarrow$ that's the digit. Beats nearest-centroid because each class now spans its natural variation.

DIAGRAM — SVD-SUBSPACE CLASSIFICATION



The digit whose SVD subspace sits closest to x (smallest perpendicular residual) wins the vote.

5.3 Tangent Distance

DEFINITION

A digit shifted, rotated, or scaled slightly is still the same digit — but Euclidean distance sees it as far away.

Tangent distance measures distance between the **small transformation subspaces** (tangent planes) of two images, so it's **invariant** to little rotations/translations/thickness changes. It dramatically lowers error on MNIST-style tasks by ignoring irrelevant deformations.

CODE — SVD-BASIS DIGIT CLASSIFIER (SKLEARN DIGITS)

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
X,y = load_digits(return_X_y=True)
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=.3,random_state=0)

k=8; bases={}
for d in range(10):
    Ad = Xtr[ytr==d].T                # pixels x samples
    U,_,_ = np.linalg.svd(Ad, full_matrices=False)
    bases[d] = U[:, :k]                # top-k basis for digit d

def predict(x):
    res=[np.linalg.norm(x - U@(U.T@x)) for d,U in bases.items()]
    return int(np.argmin(res))
acc=np.mean([predict(x)==t for x,t in zip(Xte,yte)])
print("accuracy =", round(acc,3))      # accuracy ≈ 0.96
```

REAL-WORLD

This SVD-subspace idea powers **postal ZIP-code reading**, **bank cheque digitisation**, and is the conceptual ancestor of face-recognition "eigenfaces" and modern embedding-based retrieval.

5.4 Text Mining — Latent Semantic Indexing (LSI)

💡 INTUITION

Searching text by exact keywords fails on **synonyms** ("car" vs "automobile") and **polysemy**. LSI fixes this by applying **SVD to the term–document matrix**, uncovering hidden "concepts." Documents and queries are compared in this compressed **concept space**, so semantically related items match even with no shared words.

🔗 TERM–DOCUMENT MATRIX & LSI

Build A (terms $\times$ documents), entry a_{ij} = weight (e.g. TF-IDF) of term i in doc j . Take a rank- k SVD $A \approx U_k \Sigma_k V_k^T$. Columns of V_k are documents in **concept space**; a query q is folded in as $\hat{q} = \Sigma_k^{-1} U_k^T q$. Rank documents by **cosine similarity** to $\hat{q}$.

📊 COSINE SIMILARITY (THE RANKING SCORE)

$$\text{sim}(d, q) = \langle d, q \rangle / (\|d\| \cdot \|q\|) = \cos \theta$$

1 = identical direction (very relevant), 0 = orthogonal (unrelated). Built directly from Unit 2 inner products.

📄 WORKED EXAMPLE — TINY LSI CONCEPT

Terms {car, automobile, road}, docs $D1 = \text{"car road"}$, $D2 = \text{"automobile road"}$. $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ (rows terms, cols docs). Raw keyword search for "car" would miss $D2$. After a rank-1 SVD, both docs load heavily on the same concept (vehicle+road), so a "car" query returns **both** $D1$ and $D2$ — **synonym match achieved.**

5.5 Clustering

🔗 K-MEANS & SPECTRAL CLUSTERING

Clustering groups similar documents/images without labels. **K-means** iterates: assign each point to nearest centroid, then recompute centroids — minimising within-cluster $\|x - \mu\|^2$. **Spectral clustering** uses **eigenvectors** of a similarity graph's Laplacian (Unit 2!) to cluster non-convex shapes. Often SVD first reduces dimension, then K-means clusters the concept-space vectors.

📊 K-MEANS OBJECTIVE

$$\text{minimise } J = \sum_{\text{clusters } c} \sum_{x \in c} \|x - \mu_c\|^2$$

5.6 Non-negative Matrix Factorization & LGK Bidiagonalization for Text

🔗 NMF FOR TOPICS

Apply NMF to the (non-negative) term–document matrix: $A \approx WH$. Columns of W = **topics** (lists of words), columns of H = **each document's topic mix**. Because everything is ≥ 0 , topics are additive and readable ("sports" = {game, team, score}) — unlike SVD's signed, hard-to-read factors.

Lanczos–Golub–Kahan (LGK) bidiagonalization computes a few leading singular triplets of a **huge sparse** term–document matrix without forming the full SVD — the practical engine behind large-scale LSI.

📁 CODE — LSI & NMF TOPICS ON 20-NEWSGROUPS

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD, NMF
docs = fetch_20newsgroups(subset='train',
                          categories=['sci.space', 'rec.autos']).data
A = TfidfVectorizer(stop_words='english', max_features=2000).fit_transform(docs)
lsi = TruncatedSVD(n_components=5).fit(A) # SVD -> 5 latent concepts (LSI)
nmf = NMF(n_components=5, max_iter=400).fit(A) # NMF -> 5 readable topics
print(lsi.explained_variance_ratio_.sum()) # fraction of info kept
```

⚙️ REAL-WORLD

LSI/NMF underpin **search engines**, **document deduplication**, **news topic tagging**, **spam filtering**, and **recommender systems**. They are the direct ancestors of today's word/document **embeddings** — same "project text into a low-dimensional meaning space" philosophy.

Aspect	SVD / LSI	NMF
Factors	orthogonal, signed	non-negative, additive
Interpretability	low (mixed +/-)	high (readable topics)
Optimality	best rank-k (Eckart–Young)	local optimum
Best use	retrieval, denoising	topic modelling

Table 5.1 — SVD/LSI vs NMF for text

⚠️ EXAM TRAPS — UNIT 5

- Term–document matrix: state clearly whether rows are terms or documents — the SVD factors swap roles otherwise.
- LSI compares in **concept space** via **cosine**, not raw Euclidean on word counts.
- Tangent distance is about **invariance to small transformations** — mention rotation/translation to earn the mark.
- NMF needs non-negative input; TF-IDF is fine (≥ 0), but mean-centred data is not.
- SVD-basis digit classifier scores by **projection residual**, not distance to a single mean.

🔗 Unit 5 Mini-Project

⚙️ MINI-PROJECT — SEMANTIC NEWS SEARCH ENGINE

Domain: information retrieval. **Dataset:** 20-Newsgroups (sklearn). **Task:** build a TF-IDF term–document matrix, apply TruncatedSVD (LSI), and let a user query return the top-5 most similar articles by cosine similarity; add an NMF view that prints the top words per discovered topic. **Deliverable:** a mini CLI/notebook search tool + a table of NMF topics — demonstrating synonym-robust retrieval.

Master Formula Cheat-Sheet

Topic	Formula	Use / note
Subspace test	$0 \in U$, closed under $+$ and scalar $\cdot$	3 checks
Direct sum	$U \oplus W \Leftrightarrow U \cap W = \{0\}$	unique decomposition
Dim of sum	$\dim(U+W) = \dim U + \dim W - \dim(U \cap W)$	inclusion-exclusion
Rank-nullity	$\dim V = \dim(\text{null } T) + \dim(\text{range } T)$	$n = \text{rank} + \text{nullity}$
Eigen equation	$Av = \lambda v$; $\det(A - \lambda I) = 0$	find λ then v
Diagonalization	$A = PDP^{-1}$, $A^k = PD^kP^{-1}$	eigvecs = cols of P
Norm / angle	$\ v\ = \sqrt{\langle v, v \rangle}$; $\cos \theta = \langle u, v \rangle / (\ u\ \ v\)$	orthogonal $\Leftrightarrow \langle u, v \rangle = 0$
Cauchy-Schwarz	$ \langle u, v \rangle \leq \ u\ \ v\ $	equality $\Leftrightarrow$ parallel
Riesz	$\phi(v) = \langle v, u \rangle$	functional = dot product
2x2 inverse	$1/(ad-bc) \cdot \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$	need $\det \neq 0$
Least squares	$A^T A \hat{x} = A^T b$	= linear regression
LU / QR	$A = LU$, $A = QR$	solve stably, no inverse
SVD	$A = U \Sigma V^T$, $\sigma_i = \sqrt{\text{eig}(A^T A)}$	$A_k = \sum \sigma_i u_i v_i^T$ best rank- k
Gram-Schmidt	$v_k = x_k - \sum_{j < k} (\langle x_k, v_j \rangle / \langle v_j, v_j \rangle) v_j$	build orthonormal Q
Projection	$\hat{y} = \sum (\langle y, u_j \rangle / \langle u_j, u_j \rangle) u_j$	onto orthogonal basis
K-means	$\min \sum \ x - \mu_c\ ^2$	clustering objective
Cosine sim	$\langle d, q \rangle / (\ d\ \ q\)$	LSI ranking

Glossary

Field (F): number system ($\mathbb{R}/\mathbb{C}$) with $+, -, \times, \div$.

Vector space: set closed under $+$ and scaling, 8 axioms.

Subspace: subset that is itself a vector space.

Span: all linear combinations of a set.

Linear independence: only trivial combination gives 0.

Basis: independent + spanning list.

Dimension: number of vectors in any basis.

Rank: dim of column (= row) space; # pivots.

Nullity: dim of null space; # free variables.

Eigenvalue/vector: λ, v with $Av = \lambda v$, $v \neq 0$.

Diagonalizable: $A = PDP^{-1}$ with eigenvector basis.

Inner product: $\langle u, v \rangle$; gives length & angle.

Orthogonal: $\langle u, v \rangle = 0$; orthonormal: also unit length.

Linear map: T preserving $+$ and scaling.

Null space/kernel: vectors T sends to 0.

Range/image: all outputs $T(v)$.

Isomorphism: invertible linear map; $V \cong W \Leftrightarrow$ equal dim.

Quotient V/U : cosets $v+U$; $\dim = \dim V - \dim U$.

Echelon form: staircase of pivots after row reduction.

LU/QR: lower-upper / orthogonal-upper factorizations.

SVD: $A=U\Sigma V^T$; singular values rank importance.

Least squares: minimise $\|Ax-b\|^2$; projection onto Col A.

NMF: $A\approx WH$ with $W,H\geq 0$; interpretable parts.

LSI: SVD of term–document matrix for semantic search.

Tangent distance: transformation-invariant image distance.

Condition number: σ_1/σ_n ; sensitivity of $Ax=b$.

Likely Exam Questions & Model Answers

Short-answer (2–5 marks)

 **Q1. STATE THE SUBSPACE TEST AND SHOW $\{(X,Y):X=Y\} \subseteq \mathbb{R}^2$ IS A SUBSPACE.**

A: Test = contains 0, closed under + and scaling. $0=(0,0)$ has $x=y$ ✓. If $x_1=y_1$, $x_2=y_2$ then sum has equal coords ✓; scaling keeps $x=y$ ✓. Hence a subspace (the line $y=x$). ■

 **Q2. FIND EIGENVALUES OF $A=\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$.**

A: $\det(\begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix})=(2-\lambda)^2-1=\lambda^2-4\lambda+3=0 \Rightarrow \lambda=1,3$.

 **Q3. STATE RANK–NULLITY AND APPLY TO A 3×5 MATRIX OF RANK 2.**

A: $n = \text{rank} + \text{nullity} \Rightarrow 5 = 2 + \text{nullity} \Rightarrow \text{nullity} = 3$.

 **Q4. WHY PREFER QR OVER THE NORMAL EQUATIONS FOR LEAST SQUARES?**

A: Forming $A^T A$ squares the condition number (loses accuracy for ill-conditioned A); QR solves $R\hat{x}=Q^T b$ directly, more numerically stable.

Long-answer (8–10 marks)

 **Q5. FIT THE BEST LINE TO $(1,1),(2,2),(3,2)$ BY LEAST SQUARES. SHOW ALL STEPS.**

A: $A=\begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}$, $b=(1,2,2)$. $A^T A=\begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$, $A^T b=(11,5)$. $\det=6$, $(A^T A)^{-1}=(1/6)\begin{bmatrix} 3 & -6 \\ -6 & 14 \end{bmatrix}$. $\hat{x}=(1/6)(3,4)=(0.5,0.667)$. **Line: $\hat{y} = 0.5x + 0.667$.** Residuals: at $x=1,2,3 \rightarrow \hat{y}=1.167,1.667,2.167 \rightarrow$ small errors; done. ■

 **Q6. EXPLAIN SVD AND THE ECKART–YOUNG THEOREM; HOW DOES LSI USE IT?**

A: $A=U\Sigma V^T$, U, V orthogonal, $\Sigma=\text{diag}(\sigma_i \geq 0)$. Truncating to top- k triplets gives $A_k=\sum \sigma_i u_i v_i^T$, the closest rank- k matrix to A (Eckart–Young), error $\sigma_{[k+1]}$. LSI applies SVD to the term–document matrix so documents/queries compare in a k -dimensional concept space via cosine similarity — matching synonyms and filtering noise. ■

 **Q7. PERFORM GRAM–SCHMIDT ON $X_1=(1,1,0)$, $X_2=(1,0,1)$, $X_3=(0,1,1)$.**

A: $v_1=(1,1,0)$. $v_2=(1,0,1)-\frac{1}{2}(1,1,0)=(\frac{1}{2},-\frac{1}{2},1)$. For v_3 : subtract projections on v_1 (coeff $\langle x_3, v_1 \rangle / \langle v_1, v_1 \rangle = \frac{1}{2}$) and v_2 (coeff $\langle x_3, v_2 \rangle / \langle v_2, v_2 \rangle = (-\frac{1}{2}+1)/1.5 = \frac{1}{3}$): $v_3=(0,1,1)-\frac{1}{2}(1,1,0)-\frac{1}{3}(\frac{1}{2},-\frac{1}{2},1)=(-\frac{2}{3},\frac{2}{3},\frac{2}{3})$. Orthogonal set $\{v_1, v_2, v_3\}$; normalise for orthonormal. ■

 **Q8. DIAGONALIZE $A=\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ AND COMPUTE A^5 .**

A: $\lambda=1,3$ with eigvecs $(1,-1),(1,1)$. $P=\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, $D=\text{diag}(1,3)$, $P^{-1}=\frac{1}{2}\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$. $A^5=P \cdot \text{diag}(1,243) \cdot P^{-1}=\frac{1}{2}\begin{bmatrix} 244 & 242 \\ 242 & 244 \end{bmatrix}$. ■

★ TITLE — "MATRIX METHODS STUDIO: A LOW-RANK ENGINE FOR IMAGES & TEXT"

Build one small Python application that showcases **every unit** of Computational Mathematics on real public data — proving you can take raw data → matrix → insight.

Problem

Organisations drown in high-dimensional images and documents. Show that a handful of linear-algebra tools (rank, eigen/SVD, least squares, NMF) can **compress, classify, search, and cluster** them.

Datasets (all public)

- **sklearn digits / MNIST** — images (Units 1–2, 5).
- **20-Newsgroups** — text (Units 4–5).
- **California Housing / Kaggle House Prices** — tabular (Unit 4).

What to build (mapped to units)

Module	Uses	Unit(s)
1. Data→matrix loader + rank/independence report	span, rank, basis	1, 3
2. Eigenfaces / eigen-digits + PCA variance plot	eigen, diagonalization, norms	2
3. Least-squares house-price predictor (normal eq + QR)	$Ax=b$, least squares, QR	4
4. SVD image compressor (rank-k slider)	SVD, Eckart–Young	4, 5
5. SVD-basis digit classifier (+ tangent-distance note)	subspace projection residual	5
6. LSI + NMF semantic search & topic viewer	SVD/NMF, cosine, clustering	5

Deliverables

- A Jupyter notebook / Streamlit app with the six modules.
- Figures: PCA scree plot, rank-k reconstruction grid, regression fit, NMF topic table.
- A 2-page report tying each result back to the theorem that makes it work (rank–nullity, Eckart–Young, normal equations).
- Accuracy/error numbers: digit-classifier accuracy, compression ratio vs error, search relevance.

⚙️ STACK

numpy, scipy.linalg, scikit-learn, matplotlib, optional streamlit for the UI. Everything runs on a laptop, CPU-only.

💡 WHY THIS MAKES YOU EMPLOYABLE

Finishing this capstone means you can look at **any** dataset and say: "this is a matrix — here's its rank, here's how to compress it, here's how to predict/search/cluster with it." That is exactly the mathematical maturity data-science teams hire for.

★ **YOU'VE FINISHED DS-502-MJ COMPUTATIONAL MATHEMATICS**

All 5 units · every sub-topic · intuition → definition → derivation → worked example → code → diagram →
real-world → traps.

Prepared by Sid Thaware · portfolio-sidt0518.onrender.com

Print: Chrome → Ctrl+P → Save as PDF → A4 → Margins Default → Background graphics ON.